COMP 565 Fall 2026 Course Project

PRSAgents Personalized Polygenic Risk Prediction with AI Agents

Instructor: Yue Li, School of Computer Science, McGill University. Teams: three students. Report: 6 to 8 pages. Dates: to be confirmed on myCourses.

You will construct ancestry-specific polygenic risk scores, combine them with a mixture-of-experts model in a small admixed target cohort, and build a local AI agent to refine that model. The project has three stages: source PRS and portability, personalized MoEPRS, and Ollama/Qwen refinement. Teams of three. Data and starter code are provided; no genomics data engineering or paid AI subscription is required.

1 Background and motivation

A polygenic risk score predicts a trait as a weighted sum of allele counts. In Assignment 1 you estimated those weights yourself with VIPRS. In practice you rarely start from nothing: many scores already exist for the same trait, each fit in a different population, and the European-derived score is almost always the one trained on the largest sample.

None of them works well on an individual whose genome is not European. There are two reasons, and the second one surprises people.

1. Effects can differ. A variant may act more strongly in one population than another.
2. Even identical effects transfer badly. Allele frequencies and LD patterns differ between populations, so a score fit in one population puts weight on tagging variants that tag something different, or nothing, in another.

Stage 1 uses discovery summary statistics to construct source PRS. Stages 2 and 3 reuse those fixed scores rather than refitting variant weights from the small target cohort. The central question is whether personalized combinations improve prediction and how much evidence is needed to support further refinement. Locus-specific models may be studied as optional comparators.

2 Data and simulation

Genotypes and phenotypes in this release are simulated. Founder haplotypes reproduce realistic ancestry structure and linkage disequilibrium; known generating effects allow instructor-only assessment of prediction ceilings.

2.1 Simulation design

Simulated founder haplotypes are split into disjoint source and target pools before cohort synthesis. Individuals are built as mosaics of founder haplotype blocks, preserving linkage structure within each block. Target individuals draw blocks from several ancestries.

2.2 Accessing the project materials

Download or clone the scripts, guides, and project description from github.com/yueliyl/PRSAgents. Download PRSAgents_Data.zip from the latest GitHub release and extract its data folder into the repository root, beside harness.py and README.md. The repository README gives the file guide and setup instructions; it is the current reference for the supplied files. Run all commands from the repository root after installing the Python requirements.

The 10 source scores come from five ancestry groups (EUR, AFR, EAS, SAS, AMR) crossed with two estimators (VIPRS and LASSO). The source cohorts are deliberately unequal: EUR was fit on 20,000

individuals, AMR on 1,000. The EUR score has the most reliable weights and the worst ancestry match to your target. The AMR score has the right ancestry and weights too noisy to use alone. Neither wins on its own.

3 Prediction task

You receive four anonymous generating settings, with ten replicates per setting. Each setting follows one of the four regimes below; its identity is withheld. Infer the setting from evidence where requested, and do not pass the hidden answer to the AI controller.

Regime	What varies	What structure should win
Shared	Causal effects identical across ancestries	One fixed linear combination of the sources
Ancestry-dependent	Effects differ across ancestries; admixture varies by individual	Mixing weights that depend on genetic PCs
Locus-varying	How much effects are shared differs from block to block	Source weights that vary by genomic window
Non-linear	The trait is a non-linear function of genetic liability	A non-linear genetic link within the permitted MoE family

Use the instructor-designated development dataset to check the baseline. Shared-effect simulations illustrate that poor transfer can occur even with identical causal effects. Do not expose regime labels to the agent on assessment datasets.

3.1 Project overview

Stage 1 constructs ancestry-specific PRS from discovery summary statistics and measures how they transfer across populations. Stage 2 learns a personalized mixture of those fixed scores in the small target cohort. Stage 3 uses a local Ollama/Qwen agent to investigate whether evidence supports a better gate or genetic link. Each stage produces inputs or an incumbent model for the next; the report should explain this progression.

Use the provided target split: 2,000 training, 800 validation, and 1,200 test individuals. Fit target-cohort parameters using training labels. Validation supports predeclared selection or confirmation; instructor-held test labels are never used for fitting or agent decisions. The separate Stage 1 transfer cohorts have provided phenotypes for portability assessment and are not the target test split.

4 Stage 1 Source PRS and portability

The goal is to separate two causes of poor PRS transfer: differences in causal effects, and differences in allele frequencies and linkage disequilibrium. Construct one score for each discovery source, EUR, AFR, EAS, SAS, and AMR, using summary statistics from that source and its ancestry-specific LD matrix. The core expert panel contains these five VIPRS scores. The supplied ten-score panel also includes LASSO scores; comparisons using that larger panel are optional and must be labeled separately.

Inputs and source fitting

The release provides <dataset>/stage1/sumstats.npz with per-source marginal effects, uncertainty and sample sizes, and data/stage1/ld/ld_<POP>.npz with the corresponding block-diagonal LD matrices. Use the variant/block map included with the summary statistics. Fit VIPRS with a spike-and-slab prior and mean-field variational inference. Adapt your Assignment 1 implementation, exploiting the independent LD blocks for coordinate updates. Document the prior, initialization, convergence criterion, and any source-only hyperparameter choices.

Keep source and target learning distinct. Source variant weights are estimated from discovery statistics, not from target training, validation, or test phenotypes. Check variant order, allele orientation, and score scaling before

applying the weights. Follow the released scoring conventions and document the convention used. Learn any additional target normalization using training individuals only and reuse it on validation/test and within cross-validation folds.

Transfer assessment

Use data/stage1/eval_genotypes.npz and <dataset>/stage1/eval_phenotypes.npz to assess every source PRS in every provided transfer cohort: AFR, EUR, SAS, EAS, NAT, and admixed AMR. These six cohorts contain 1,500 individuals each. Produce a five-row by six-column transfer matrix using incremental R^2 above the provided covariates. State which covariates are available in these transfer files; do not silently substitute the target cohort covariates.

Across assigned replicates, report mean transfer accuracy and uncertainty. Discuss ancestry-matched versus off-diagonal entries, the EUR score's accuracy in AFR relative to its accuracy in EUR, unequal discovery sizes, and the absence of a NAT discovery expert. Compare shared-effect and heterogeneous-effect settings where their identities are supplied for analysis; keep assessment regime labels out of the Stage 3 model context. Poor transfer in the shared-effect setting is evidence that effect divergence is not the only mechanism.

Reference and deliverables

Use stages/stage1_fit.py as the input-loading and transfer-table reference. From the project directory, this command runs the reference check; replace its example path with the assigned dataset:

```
python stages/stage1_fit.py data/datasets/dataset_01/rep00
```

Submit the source-fitting implementation, five variant-weight vectors per dataset/replicate, the five fixed source-score columns for target individuals, transfer matrices with uncertainty, and an interpretation of portability. Supplied VIPRS scores are a fallback for progressing to later stages; identify their use explicitly. Running the fallback alone does not replace the Stage 1 implementation and analysis deliverable.

5 Stage 2 Personalized mixture of experts

The goal is to test whether a PC-dependent combination of existing PRS improves on one source score, a global combination, or refitting a model on target genotypes. Use the five Stage 1 VIPRS scores as fixed experts. Hold this panel constant in all core comparisons and carry it into Stage 3.

Model and constraints

For individual i, S_{ik} is the standardized score from source k, g_i contains PC1 to PC10, and c_i contains age, sex, and PC1 to PC10. Fit the following conditional-mean prediction:

$$\hat{y}_i = a_0 + c_i^T \alpha + \sum_{k=1}^{K} \pi_k(g_i)\gamma_k S_{ik}$$

$$\pi(g_i) = \mathrm{softmax}\left(\theta^T \begin{bmatrix} 1 \\ g_i \end{bmatrix}\right)$$

The global intercept/covariate coefficients, one scalar γ_k per expert, gate coefficients θ, and residual variance are learned in the target cohort. Source variant weights remain fixed. The linear softmax gate reads genetic PCs only; age and sex cannot enter it. Global covariate adjustment and ancestry-dependent source weighting serve different roles. Gate weights must be finite, nonnegative, and sum to one for each individual.

Use the provided implementation

Use the provided compact implementation in moeprs.py; deriving EM or implementing it from scratch is not required. Start with stage2_demo.py, then run stage2_baselines.py for the required comparisons. Explain in your own words how the E-step assigns soft responsibilities using gate probabilities and phenotype fit, and how the M-step updates expert calibration, global covariate effects, residual variance, and the gate. Distinguish training responsibilities from the gate probabilities used to predict a new individual.

The default teaching model uses Gaussian experts with a shared residual variance, scalar expert calibration, global covariate adjustment, and a linear PC-only softmax gate. It includes numerical safeguards and records a training likelihood trace. Inspect finite predictions, gate weights summing to one, and fitting behavior. Stage 2 uses ten gate PCs and an L2 gate penalty of 0.01. Freeze the incumbent before Stage 3 and use the same implementation and settings for both search controllers.

```
python stage2_demo.py data/datasets/dataset_01/rep00
python stage2_baselines.py data/datasets/dataset_01/rep00
```

Alternatively, use the implementation accompanying the paper at https://github.com/shz9/moe-prs-paper (model/moe.py and its example notebooks). Record its commit and settings, adapt the supplied data to its API, and retain the same fixed expert panel, PC-only gate, global covariates, and split rules. The compact model is an independent teaching version with fewer options; identical numerical results are not expected. STAGE2_GUIDE.md explains both routes. Using the paper backend in Stage 3 requires adapting both controllers to it.

Baselines and interpretation

Fit the required baselines on the same target training individuals and with the same covariates: best single PRS, MultiPRS linear stacking with global coefficients, and ridge regression on the 1,000 typed target variants. Choose the single PRS and tune ridge penalties using a declared training-CV or validation protocol, never test labels. For ridge, fit genotype scaling on training individuals and apply it unchanged to validation/test.

Report incremental R^2 and frozen-prediction error for each method, with paired differences across assigned replicates. Plot the fitted gate weights against PC1 and PC2 and explain whether they vary with ancestry structure while retaining mixtures of experts. A positive gain over MultiPRS supports personalization; a gain over genotype ridge supports reuse at this target sample size. Explain unfavorable or uncertain comparisons rather than treating a win as guaranteed.

Repeat Stage 2 with nested random training subsets of 250, 500, 1,000, and 2,000 individuals, or all feasible sizes if a released split is smaller. Keep the validation/test individuals fixed across sizes, use identical subsets for every method, and refit preprocessing and tune models using only the allowed labels at each size. Report paired learning curves showing where gating pays and where reuse beats direct refitting. Do not choose the final operating point using test results.

Deliverables and handoff to Stage 3

Submit the declared implementation and configuration, a brief explanation of the model and EM steps, fitting checks, a baseline table with paired uncertainty, gate-weight plots, the sample-size sweep, and frozen Stage 2 validation/test predictions. A formal EM derivation and from-scratch implementation are optional. Pass fixed source scores, the Stage 2 incumbent, and diagnostic/evaluation interfaces to Stage 3. Its twelve-candidate allowance includes the cross-validated Stage 2 baseline; source fitting and the separate Stage 2 study are outside that allowance.

6 Stage 3 AI agent refinement with Ollama and Qwen

Use an actual LLM agent to propose, evaluate, and refine MoEPRS. Run Qwen locally through Ollama. The LLM interprets diagnostics and chooses experiments; Python fits the models and returns evidence. Local

execution requires no paid subscription or API key. A rule-based controller or grid search is a comparison, not a substitute for the required AI agent.

Install and test the local model

Install Ollama from https://ollama.com/download and open it. On Linux, run ollama serve if the server is not already running. Download the model:

```
ollama pull qwen3:8b
```

From the project directory, create and activate a Python virtual environment, then install the packages:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

On Windows, use python -m venv .venv and .venv\Scripts\Activate.ps1 in PowerShell. Follow stages/STAGE3_OLLAMA.md to make your first Python model call. Downloads need internet access; inference runs locally. Check memory and runtime early. Contact the instructor if your hardware cannot complete the setup. A smaller local Qwen model requires instructor agreement and must be identified separately in the results.

Run the starter agent

Run the supplied script from the project directory:

```
python qwen_prs_agent.py --project .
--data data/datasets/dataset_01/rep00 --model qwen3:8b
--budget 6 --out outputs/qwen_demo
```

The command above is shown wrapped for readability; enter it on one line. Replace the dataset path with your assigned path. The starter uses the five supplied VIPRS experts, evaluates a Stage 2 baseline, and lets Qwen call evaluate_variant and select_variant. Inspect trace.json and predictions.csv in the output directory. Identify real model/tool calls, hypotheses, returned evidence, and final selection. The six-evaluation demonstration uses one validation split and is not the final experiment protocol.

Preserve the MoE constraints

Hold source PRS weights fixed and use the same expert panel for all compared methods. Keep a softmax gate whose weights sum to one and whose inputs are genetic PCs or transformations of PCs. Age and sex remain global additive effects. Do not replace MoEPRS with boosting, an unrestricted neural network, or a predictor trained on raw genotypes. Document whether your fixed scores were fitted in Stage 1 or supplied.

Implement gate PC/penalty choices and at least one nonlinear family: a small regularized nonlinear PC basis, monotone expert links, or an aggregate monotone spline. The aggregate spline is an explicit permitted extension applied only to the genetic mixture score, before adding covariate effects. Keep an identity option and regularize departures from Stage 2.

Build the autonomous tool loop

The LLM must read diagnostics and experiment history, state a hypothesis, call registered fitting/evaluation tools, interpret their results, and choose another experiment or finalize. Expose diagnosis, candidate evaluation, experiment history, and selection, separately or within the evaluation tool. Validate arguments in Python, cache duplicate experiments, handle tool errors, and cap the number of model turns. Multiple agents and arbitrary generated-code execution are not required.

Use qwen3:8b with think=False and temperature 0 for the main comparison. Record the model digest from ollama list, Ollama and package versions, seeds, hardware, runtime, and complete messages/tool results. Fixed

settings do not guarantee identical generations. The agent must respond to evidence; a fixed sweep followed by an AI-written narrative is insufficient.

Evaluate changes fairly

Extend the starter to three-fold cross-validation within the training split. All candidates and Stage 2 use the same folds and individuals. Fit preprocessing only on fitting folds and score held-out predictions. Return incremental R^2, frozen-prediction MSE, paired squared-error changes relative to Stage 2, uncertainty, and ancestry-region diagnostics. Positive baseline squared error minus candidate squared error indicates improvement. Residual curvature motivates a link correction; PRS-by-PC residual associations are more specific evidence of a gate defect than residual-PC correlation alone.

Allow at most twelve candidate evaluations per dataset per controller, including Stage 2. Each evaluation contains three fold fits, so twelve candidates need up to 36 fits. Failed fits count toward the candidate allowance; cached duplicates do not. Log all additional calibration/inner fits and model calls separately. Fit any spline or calibration without using the labels of the individuals being scored.

Choose one candidate from cross-validation before inspecting validation. Fit it and Stage 2 on the full training split and confirm once on validation. Predeclare a confirmation rule; for example, retain Stage 2 if the candidate has worse validation MSE. Do not rerank candidates or reopen search. These two final fits and validation assessments are outside the twelve-candidate search allowance and available equally to both controllers. Keep the chosen training fit for test predictions.

Uncertainty conditional on frozen predictions is descriptive and does not account for fitting or adaptive selection. Folds are not independent datasets. Report paired differences across independent replicates on common support. The existing incremental-R^2 helper recalibrates using evaluation labels; also report error from frozen predictions and explain this distinction. A maximum selected on test is not an unbiased estimate of recoverable gain.

Compare and submit

Compare Stage 2, the Ollama/Qwen agent, and fixed or random search over the same permitted candidate space. Give both search controllers identical candidate budgets, folds, and confirmation rules. Use development runs to debug and then freeze prompts, tools, settings, candidate space, and selection rules before final assessment. Do not expose test labels, hidden regime information, or instructor oracle metadata to the LLM.

Submit runnable code and setup instructions, complete traces, a table of cross-validation and confirmation results with paired uncertainty, evaluation/fit/runtime logs, and test predictions for all three methods on every assigned dataset and replicate. Explain whether the AI decisions improved performance or efficiency and why each retained change was justified. A well-supported negative result or justified retention of Stage 2 receives full credit for the investigation. See stages/STAGE3_OLLAMA.md for the full protocol and official Ollama references.

7 Required baselines

The required core comparison is best single PRS, MultiPRS, genotype ridge, and Stage 2 MoEPRS on every assigned dataset. The remaining methods are optional comparators. Stage 3 additionally requires fixed or random search under the same budget as the AI controller.

Baseline	Status and purpose
Best single PRS	Required; source chosen without test labels
AMR PRS alone	Optional; compare discovery size and ancestry match
MultiPRS linear stacking	Required; global mixture over the same fixed experts
Stage 2 MoEPRS	Required; personalized PC-only gate and Stage 3 incumbent

Baseline	Status and purpose
Window-weighted model	Optional; locus-specific comparator
Gradient boosting on scores	Optional comparator outside Stage 3 MoE search
Off-the-shelf AutoML	Optional comparator outside Stage 3 MoE search
Ridge on raw genotype	Required; direct refitting on the same training split

8 Oracle assessment

Because the phenotype is simulated, the instructor can assess the oracle genetic prediction. Target test labels and oracle metadata are not included in the student package. Analyze oracle comparisons when instructor feedback is supplied after submission; do not estimate a search ceiling by choosing the best candidate on test.

Discuss source-score limitations, model restrictions, fitting error, and selection uncertainty separately. The gap to an oracle is not attributable to search alone. If a model-class comparator is supplied, compare on common individuals and report paired uncertainty without using that feedback to retune the agent.

9 Evaluation and submission

The primary metric is **incremental** R^2 on the held-out test split: R^2 of covariates plus your predictor, minus R^2 of covariates alone. Also report:

1. R^2 within each tercile of admixture proportion. A method that gains on average while losing in the least European tercile is making the original problem worse, and you should say so.
2. Calibration slope from regressing observed on predicted.
3. Validation R^2 and test R^2 side by side. The gap will be larger than you expect. That gap is a finding, not an embarrassment.

Submit frozen test predictions for instructor assessment. Test feedback must not enter the Stage 3 search or be used to revise its policy. Run all assigned simulation replicates; when ten per regime are provided, report paired means and standard errors over those ten. Distinguish validation results from test results supplied by the instructor.

10 Deliverables

1. Code repository reproducing all three stages: source fitting, weights and target scores; calls to the provided or paper Stage 2 model and baselines; the sample-size sweep; and Stage 3 agent/comparison runs.
2. Complete Ollama/Qwen message and tool-call traces for every assigned run, showing hypotheses, results, revisions, final selection, and any fallback. Include prompts, model digest/settings, budgets, fit counts, and runtime.
3. Report, 6 to 8 pages: Stage 1 transfer matrices and portability analysis; Stage 2 baseline contrasts, gate plots, and sample-size sweep; Stage 3 agent/comparison results and interpretation; oracle assessment and regime identification where applicable.
4. Presentation, 10 minutes plus questions.

11 Grading

Component	Weight
Experimental hygiene: correct splits, replicates with error bars, no leakage	20
Agent design: tool set, loop, budget discipline, quality of the trace	25
Baselines and oracle ladder, run and interpreted	15

Component	Weight
Regime identification and analysis	20
Report and presentation	20

A well-executed negative result scores in the same band as a positive one. If your agent does not beat the tuned baselines and your report explains clearly where it lost, that is full credit. An agent that wins without an explanation is not.

12 Timeline

Week	Milestone
1	Stage 1 inputs and source fitting checked; transfer table reproduced
2	Provided Stage 2 model and baselines run; gate plots and size sweep
3	Local Qwen tool loop working end to end; setup trace submitted
4	Stage 3 CV and confirmation protocol; assigned assessment runs
5	Controller ablation, paired replicate contrasts, and interpretation
6	Complete three-stage report and presentation

Exact dates to be confirmed on myCourses.

13 Getting started

After downloading the data as described in Section 2.2, run all commands from the repository root after activating its Python environment. Begin with the Stage 1 source-fitting reference and transfer table, then read STAGE2_GUIDE.md and run stage2_demo.py followed by stage2_baselines.py. Use the supplied Stage 2 model unchanged if desired. Follow stages/STAGE3_OLLAMA.md to install local Qwen and run the tool-calling demonstration. Replace example dataset paths with assigned release paths. Bring setup questions to office hours early.